

DevOps Cheat Sheet

Linux, Git, Docker, Kubernetes, Helm, Terraform

Every command, grouped by the tool and by what you are trying to do.

Bhuvnesh Verma

github.com/MasterBhuvnesh/notebooks

Contents

1	Linux	2
1.1	Basic	2
1.2	Intermediate	2
1.3	Advanced	2
1.4	Networking	3
1.5	File management and search	3
1.6	System monitoring	3
1.7	Package management	4
1.8	Disk and filesystem	4
1.9	Scripting and automation	4
1.10	Development and debugging	4
1.11	Other useful commands	4
2	Git	6
2.1	Basic	6
2.2	Branching and merging	6
2.3	Remote repositories	6
2.4	Stashing and cleaning	7
2.5	Tagging	7
2.6	Advanced	7
2.7	GitHub CLI	7
3	Docker	9
3.1	Basic	9
3.2	Intermediate	9
3.3	Advanced	10
4	Kubernetes	11
4.1	Basic	11
4.2	Intermediate	11
4.3	Advanced	11
5	Helm	13
5.1	Basic	13
5.2	Installing and upgrading charts	13
5.3	Working with charts	13
5.4	Inspecting and testing	14
5.5	Values and customisation	14
5.6	Namespaces and contexts	14
5.7	Repositories and publishing	15
5.8	History and rollbacks	15
6	Terraform	16

1. Linux

Linux is the ground every other tool here stands on. These commands navigate the filesystem, manage files, configure permissions and automate work from a terminal.

1.1 Basic

Command	Description
pwd	Print the current working directory
ls	List files and directories
cd	Change directory
touch	Create an empty file
mkdir	Create a new directory
rm	Remove files or directories
rmdir	Remove empty directories
cp	Copy files or directories
mv	Move or rename files and directories
cat	Display the content of a file
echo	Display a line of text
clear	Clear the terminal screen

1.2 Intermediate

Command	Description
chmod	Change file permissions
chown	Change file ownership
find	Search for files and directories
grep	Search for text in a file
wc	Count lines, words and characters in a file
head	Display the first few lines of a file
tail	Display the last few lines of a file
sort	Sort the contents of a file
uniq	Remove duplicate lines from a file
diff	Compare two files line by line
tar	Archive files into a tarball
zip/unzip	Compress and extract ZIP files
df	Display disk space usage
du	Display directory size
top	Monitor system processes in real time
ps	Display active processes
kill	Terminate a process by its PID
ping	Check network connectivity
wget	Download files from the internet
curl	Transfer data from or to a server
scp	Securely copy files between systems
rsync	Synchronise files and directories

1.3 Advanced

Command	Description
awk	Text processing and pattern scanning
sed	Stream editor for filtering and transforming text
cut	Remove sections from each line of a file
tr	Translate or delete characters
xargs	Build and execute command lines from standard input
ln	Create symbolic or hard links
df -h	Display disk usage in human-readable format
free	Display memory usage
iostat	Display CPU and I/O statistics
netstat	Network statistics; ss is the modern alternative
ifconfig/ip	Configure network interfaces; ip is the modern alternative
iptables	Configure firewall rules
systemctl	Control the systemd system and service manager
journalctl	View system logs
crontab	Schedule recurring tasks
at	Schedule tasks for a specific time
uptime	Display system uptime
whoami	Display the current user
users	List all users currently logged in
hostname	Display or set the system hostname
env	Display environment variables
export	Set environment variables

1.4 Networking

Command	Description
ip addr	Display or configure IP addresses
ip route	Show or manipulate routing tables
traceroute	Trace the route packets take to a host
nslookup	Query DNS records
dig	Query DNS servers
ssh	Connect to a remote server via SSH
ftp	Transfer files using the FTP protocol
nmap	Network scanning and discovery
telnet	Communicate with remote hosts
netcat (nc)	Read and write data over networks

1.5 File management and search

Command	Description
locate	Find files quickly using a database
stat	Display detailed information about a file
tree	Display directories as a tree
file	Determine a file's type
basename	Extract the filename from a path
dirname	Extract the directory part of a path

1.6 System monitoring

Command	Description
<code>vmstat</code>	Display virtual memory statistics
<code>htop</code>	Interactive process viewer, an alternative to <code>top</code>
<code>lsof</code>	List open files
<code>dmesg</code>	Print kernel ring buffer messages
<code>uptime</code>	Show how long the system has been running
<code>iostat</code>	Display real-time disk I/O by process

1.7 Package management

Command	Description
<code>apt</code>	Package manager for Debian-based distributions
<code>yum/dnf</code>	Package manager for RHEL-based distributions
<code>snap</code>	Manage snap packages
<code>rpm</code>	Manage RPM packages

1.8 Disk and filesystem

Command	Description
<code>mount/umount</code>	Mount or unmount filesystems
<code>fsck</code>	Check and repair filesystems
<code>mkfs</code>	Create a new filesystem
<code>blkid</code>	Display information about block devices
<code>lsblk</code>	List information about block devices
<code>parted</code>	Manage partitions interactively

1.9 Scripting and automation

Command	Description
<code>bash</code>	Command interpreter and scripting shell
<code>sh</code>	Legacy shell interpreter
<code>cron</code>	Automate recurring tasks
<code>alias</code>	Create shortcuts for commands
<code>source</code>	Execute commands from a file in the current shell

1.10 Development and debugging

Command	Description
<code>gcc</code>	Compile C programs
<code>make</code>	Build and manage projects
<code>strace</code>	Trace system calls and signals
<code>gdb</code>	Debug programs
<code>git</code>	Version control system
<code>vim/nano</code>	Text editors for scripting and editing

1.11 Other useful commands

Command	Description
<code>uptime</code>	Display system uptime
<code>date</code>	Display or set the system date and time
<code>cal</code>	Display a calendar
<code>man</code>	Display the manual for a command
<code>history</code>	Show previously executed commands
<code>alias</code>	Create custom shortcuts for commands

2. Git

Git tracks every change, lets a team work on the same code without collisions, and lets you undo a mistake. These commands cover the everyday work and the recovery tools you reach for when something has gone wrong.

2.1 Basic

Command	Description	Example
<code>git init</code>	Initialises a new Git repository in the current directory	<code>git init</code>
<code>git clone</code>	Copies a remote repository to the local machine	<code>git clone https://github.com/user/repo.git</code>
<code>git status</code>	Displays the state of the working directory and staging area	<code>git status</code>
<code>git add</code>	Adds changes to the staging area	<code>git add file.txt</code>
<code>git commit</code>	Records changes to the repository	<code>git commit -m "Initial commit"</code>
<code>git config</code>	Configures user settings, such as name and email	<code>git config --global user.name "Your Name"</code>
<code>git log</code>	Shows the commit history	<code>git log</code>
<code>git show</code>	Displays detailed information about a specific commit	<code>git show <commit-hash></code>
<code>git diff</code>	Shows changes between commits, the working directory and the staging area	<code>git diff</code>
<code>git reset</code>	Unstages changes or resets commits	<code>git reset HEAD file.txt</code>

2.2 Branching and merging

Command	Description	Example
<code>git branch</code>	Lists branches or creates a new branch	<code>git branch feature-branch</code>
<code>git checkout</code>	Switches between branches or restores files	<code>git checkout feature-branch</code>
<code>git switch</code>	Switches branches, the modern alternative to <code>git checkout</code>	<code>git switch feature-branch</code>
<code>git merge</code>	Combines changes from one branch into another	<code>git merge feature-branch</code>
<code>git rebase</code>	Moves or combines commits from one branch onto another	<code>git rebase main</code>
<code>git cherry-pick</code>	Applies specific commits from one branch to another	<code>git cherry-pick <commit-hash></code>

2.3 Remote repositories

Command	Description	Example
<code>git remote</code>	Manages remote repository connections	<code>git remote add origin https://github.com/user/repo.git</code>

Command	Description	Example
<code>git push</code>	Sends changes to a remote repository	<code>git push origin main</code>
<code>git pull</code>	Fetches and merges changes from a remote repository	<code>git pull origin main</code>
<code>git fetch</code>	Downloads changes from a remote repository without merging	<code>git fetch origin</code>
<code>git remote -v</code>	Lists the URLs of remote repositories	<code>git remote -v</code>

2.4 Stashing and cleaning

Command	Description	Example
<code>git stash</code>	Temporarily saves changes not yet committed	<code>git stash</code>
<code>git stash pop</code>	Applies stashed changes and removes them from the stash list	<code>git stash pop</code>
<code>git stash list</code>	Lists all stashes	<code>git stash list</code>
<code>git clean</code>	Removes untracked files from the working directory	<code>git clean -f</code>

2.5 Tagging

Command	Description	Example
<code>git tag</code>	Creates a tag for a specific commit	<code>git tag -a v1.0 -m "Version 1.0"</code>
<code>git tag -d</code>	Deletes a tag	<code>git tag -d v1.0</code>
<code>git push --tags</code>	Pushes tags to a remote repository	<code>git push origin --tags</code>

2.6 Advanced

Command	Description	Example
<code>git bisect</code>	Finds the commit that introduced a bug	<code>git bisect start</code>
<code>git blame</code>	Shows which commit and author modified each line of a file	<code>git blame file.txt</code>
<code>git reflog</code>	Shows a log of changes to the tip of branches	<code>git reflog</code>
<code>git submodule</code>	Manages external repositories as submodules	<code>git submodule add https://github.com/user/repo.git</code>
<code>git archive</code>	Creates an archive of the repository files	<code>git archive --format=zip HEAD > archive.zip</code>
<code>git gc</code>	Cleans up unnecessary files and optimises the repository	<code>git gc</code>

2.7 GitHub CLI

Command	Description	Example
<code>gh auth login</code>	Logs into GitHub via the command line	<code>gh auth login</code>
<code>gh repo clone</code>	Clones a GitHub repository	<code>gh repo clone user/repo</code>
<code>gh issue list</code>	Lists issues in a GitHub repository	<code>gh issue list</code>
<code>gh pr create</code>	Creates a pull request on GitHub	<code>gh pr create --title "New Feature"</code> <code>--body "Description"</code>
<code>gh repo create</code>	Creates a new GitHub repository	<code>gh repo create my-repo</code>

3. Docker

Docker packages an application and everything it needs into a portable container, so it runs the same way on a laptop, in CI and in production.

3.1 Basic

Command	Description	Example
<code>docker --version</code>	Displays the installed Docker version	<code>docker --version</code>
<code>docker info</code>	Shows system-wide information, such as the number of containers and images	<code>docker info</code>
<code>docker pull</code>	Downloads an image from a registry, Docker Hub by default	<code>docker pull ubuntu:latest</code>
<code>docker images</code>	Lists all downloaded images	<code>docker images</code>
<code>docker run</code>	Creates and starts a new container from an image	<code>docker run -it ubuntu bash</code>
<code>docker ps</code>	Lists running containers	<code>docker ps</code>
<code>docker ps -a</code>	Lists all containers, including stopped ones	<code>docker ps -a</code>
<code>docker stop</code>	Stops a running container	<code>docker stop container_name</code>
<code>docker start</code>	Starts a stopped container	<code>docker start container_name</code>
<code>docker rm</code>	Removes a container	<code>docker rm container_name</code>
<code>docker rmi</code>	Removes an image	<code>docker rmi image_name</code>
<code>docker exec</code>	Runs a command inside a running container	<code>docker exec -it container_name bash</code>

3.2 Intermediate

Command	Description	Example
<code>docker build</code>	Builds an image from a Dockerfile	<code>docker build -t my_image .</code>
<code>docker commit</code>	Creates a new image from a container's changes	<code>docker commit container_name my_image:tag</code>
<code>docker logs</code>	Fetches logs from a container	<code>docker logs container_name</code>
<code>docker inspect</code>	Returns detailed information about a container or image	<code>docker inspect container_name</code>
<code>docker stats</code>	Displays live resource usage of running containers	<code>docker stats</code>
<code>docker cp</code>	Copies files between a container and the host	<code>docker cp container_name:/path/in/container /path/on/host</code>
<code>docker rename</code>	Renames a container	<code>docker rename old_name new_name</code>
<code>docker network ls</code>	Lists all Docker networks	<code>docker network ls</code>
<code>docker network create</code>	Creates a new Docker network	<code>docker network create my_network</code>
<code>docker network inspect</code>	Shows details about a Docker network	<code>docker network inspect my_network</code>
<code>docker network connect</code>	Connects a container to a network	<code>docker network connect my_network container_name</code>
<code>docker volume ls</code>	Lists all Docker volumes	<code>docker volume ls</code>
<code>docker volume create</code>	Creates a new Docker volume	<code>docker volume create my_volume</code>
<code>docker volume inspect</code>	Provides details about a volume	<code>docker volume inspect my_volume</code>

Command	Description	Example
<code>docker volume rm</code>	Removes a Docker volume	<code>docker volume rm my_volume</code>

3.3 Advanced

Command	Description	Example
<code>docker-compose up</code>	Starts services defined in a <code>docker-compose.yml</code> file	<code>docker-compose up</code>
<code>docker-compose down</code>	Stops and removes services defined in a <code>docker-compose.yml</code> file	<code>docker-compose down</code>
<code>docker-compose logs</code>	Displays logs for services managed by Docker Compose	<code>docker-compose logs</code>
<code>docker-compose exec</code>	Runs a command in a service's container	<code>docker-compose exec service_name bash</code>
<code>docker save</code>	Exports an image to a tar file	<code>docker save -o my_image.tar my_image:tag</code>
<code>docker load</code>	Imports an image from a tar file	<code>docker load < my_image.tar</code>
<code>docker export</code>	Exports a container's filesystem as a tar file	<code>docker export container_name > container.tar</code>
<code>docker import</code>	Creates an image from an exported container	<code>docker import container.tar my_new_image</code>
<code>docker system df</code>	Displays disk usage by Docker objects	<code>docker system df</code>
<code>docker system prune</code>	Cleans up unused images, containers, volumes and networks	<code>docker system prune</code>
<code>docker tag</code>	Assigns a new tag to an image	<code>docker tag old_image_name new_image_name</code>
<code>docker push</code>	Uploads an image to a registry	<code>docker push my_image:tag</code>
<code>docker login</code>	Logs into a Docker registry	<code>docker login</code>
<code>docker logout</code>	Logs out of a Docker registry	<code>docker logout</code>
<code>docker swarm init</code>	Initialises a Docker Swarm mode cluster	<code>docker swarm init</code>
<code>docker service create</code>	Creates a new service in Swarm mode	<code>docker service create --name my_service nginx</code>
<code>docker stack deploy</code>	Deploys a stack using a Compose file in Swarm mode	<code>docker stack deploy -c docker-compose.yml my_stack</code>
<code>docker stack rm</code>	Removes a stack in Swarm mode	<code>docker stack rm my_stack</code>
<code>docker checkpoint create</code>	Creates a checkpoint for a container	<code>docker checkpoint create container_name checkpoint_name</code>
<code>docker checkpoint ls</code>	Lists checkpoints for a container	<code>docker checkpoint ls container_name</code>
<code>docker checkpoint rm</code>	Removes a checkpoint	<code>docker checkpoint rm container_name checkpoint_name</code>

4. Kubernetes

Kubernetes is the conductor of a container orchestra. It automates the deployment, scaling and management of containerised applications across a cluster of machines.

4.1 Basic

Command	Description	Example
<code>kubectl version</code>	Displays the Kubernetes client and server version	<code>kubectl version --short</code>
<code>kubectl cluster-info</code>	Shows information about the cluster	<code>kubectl cluster-info</code>
<code>kubectl get nodes</code>	Lists all nodes in the cluster	<code>kubectl get nodes</code>
<code>kubectl get pods</code>	Lists all pods in the default namespace	<code>kubectl get pods</code>
<code>kubectl get services</code>	Lists all services in the default namespace	<code>kubectl get services</code>
<code>kubectl get namespaces</code>	Lists all namespaces in the cluster	<code>kubectl get namespaces</code>
<code>kubectl describe pod</code>	Shows detailed information about a specific pod	<code>kubectl describe pod pod-name</code>
<code>kubectl logs</code>	Displays logs for a specific pod	<code>kubectl logs pod-name</code>
<code>kubectl create namespace</code>	Creates a new namespace	<code>kubectl create namespace my-namespace</code>
<code>kubectl delete pod</code>	Deletes a specific pod	<code>kubectl delete pod pod-name</code>

4.2 Intermediate

Command	Description	Example
<code>kubectl apply</code>	Applies changes defined in a YAML file	<code>kubectl apply -f deployment.yaml</code>
<code>kubectl delete</code>	Deletes resources defined in a YAML file	<code>kubectl delete -f deployment.yaml</code>
<code>kubectl scale</code>	Scales a deployment to the desired number of replicas	<code>kubectl scale deployment my-deployment --replicas=3</code>
<code>kubectl expose</code>	Exposes a pod or deployment as a service	<code>kubectl expose deployment my-deployment --type=LoadBalancer --port=80</code>
<code>kubectl exec</code>	Executes a command in a running pod	<code>kubectl exec -it pod-name -- /bin/bash</code>
<code>kubectl port-forward</code>	Forwards a local port to a port in a pod	<code>kubectl port-forward pod-name 8080:80</code>
<code>kubectl get configmaps</code>	Lists all ConfigMaps in the namespace	<code>kubectl get configmaps</code>
<code>kubectl get secrets</code>	Lists all Secrets in the namespace	<code>kubectl get secrets</code>
<code>kubectl edit</code>	Edits a resource definition directly in the editor	<code>kubectl edit deployment my-deployment</code>
<code>kubectl rollout status</code>	Displays the status of a deployment rollout	<code>kubectl rollout status deployment/my-deployment</code>

4.3 Advanced

Command	Description	Example
kubectl rollout undo	Rolls back a deployment to a previous revision	kubectl rollout undo deployment/my-deployment
kubectl top nodes	Shows resource usage for nodes	kubectl top nodes
kubectl top pods	Displays resource usage for pods	kubectl top pods
kubectl cordon	Marks a node as unschedulable	kubectl cordon node-name
kubectl uncordon	Marks a node as schedulable	kubectl uncordon node-name
kubectl drain	Safely evicts all pods from a node	kubectl drain node-name --ignore-daemonsets
kubectl taint	Adds a taint to a node to control pod placement	kubectl taint nodes node-name key=value:NoSchedule
kubectl get events	Lists all events in the cluster	kubectl get events
kubectl apply -k	Applies resources from a kustomization directory	kubectl apply -k ./kustomization-dir/
kubectl config view	Displays the kubeconfig file	kubectl config view
kubectl config use-context	Switches the active context in kubeconfig	kubectl config use-context my-cluster
kubectl debug	Creates a debugging session for a pod	kubectl debug pod-name
kubectl delete namespace	Deletes a namespace and its resources	kubectl delete namespace my-namespace
kubectl patch	Updates a resource using a patch	kubectl patch deployment my-deployment -p '{"spec": {"replicas": 2}}'
kubectl rollout history	Shows the rollout history of a deployment	kubectl rollout history deployment my-deployment
kubectl autoscale	Automatically scales a deployment based on resource usage	kubectl autoscale deployment my-deployment --cpu-percent=50 --min=1 --max=10
kubectl label	Adds or modifies a label on a resource	kubectl label pod pod-name environment=production
kubectl annotate	Adds or modifies an annotation on a resource	kubectl annotate pod pod-name description="My app pod"
kubectl delete pv	Deletes a PersistentVolume	kubectl delete pv my-pv
kubectl get ingress	Lists all Ingress resources in the namespace	kubectl get ingress
kubectl create configmap	Creates a ConfigMap from a file or literal values	kubectl create configmap my-config --from-literal=key1=value1
kubectl create secret	Creates a Secret from a file or literal values	kubectl create secret generic my-secret --from-literal=password=myPassword
kubectl api-resources	Lists all available API resources in the cluster	kubectl api-resources
kubectl api-versions	Lists all API versions supported by the cluster	kubectl api-versions
kubectl get crds	Lists all CustomResourceDefinitions	kubectl get crds

5. Helm

Helm is the package manager for Kubernetes. It installs and manages applications packaged as charts, the way `apt` installs packages on Debian.

5.1 Basic

Command	Description	Example
<code>helm help</code>	Displays help for the Helm CLI or a specific command	<code>helm help</code>
<code>helm version</code>	Shows the Helm client and server version	<code>helm version</code>
<code>helm repo add</code>	Adds a new chart repository	<code>helm repo add stable https://charts.helm.sh/stable</code>
<code>helm repo update</code>	Updates all chart repositories to the latest version	<code>helm repo update</code>
<code>helm repo list</code>	Lists all the repositories added to Helm	<code>helm repo list</code>
<code>helm search hub</code>	Searches for charts on Helm Hub	<code>helm search hub nginx</code>
<code>helm search repo</code>	Searches for charts in the repositories	<code>helm search repo stable/nginx</code>
<code>helm show chart</code>	Displays a chart's metadata and dependencies	<code>helm show chart stable/nginx</code>

5.2 Installing and upgrading charts

Command	Description	Example
<code>helm install</code>	Installs a chart into a Kubernetes cluster	<code>helm install my-release stable/nginx</code>
<code>helm upgrade</code>	Upgrades an existing release with a new version of the chart	<code>helm upgrade my-release stable/nginx</code>
<code>helm upgrade --install</code>	Installs a chart if it is not installed, upgrades it if it is	<code>helm upgrade --install my-release stable/nginx</code>
<code>helm uninstall</code>	Uninstalls a release	<code>helm uninstall my-release</code>
<code>helm list</code>	Lists all the releases installed on the cluster	<code>helm list</code>
<code>helm status</code>	Displays the status of a release	<code>helm status my-release</code>

5.3 Working with charts

Command	Description	Example
<code>helm create</code>	Creates a new Helm chart in a specified directory	<code>helm create my-chart</code>
<code>helm lint</code>	Lints a chart to check for common errors	<code>helm lint ./my-chart</code>
<code>helm package</code>	Packages a chart into a <code>.tgz</code> file	<code>helm package ./my-chart</code>
<code>helm template</code>	Renders the Kubernetes YAML from a chart without installing it	<code>helm template my-release ./my-chart</code>
<code>helm dependency update</code>	Updates the dependencies in <code>Chart.yaml</code>	<code>helm dependency update ./my-chart</code>

Command	Description	Example
<code>helm dependency build</code>	Builds dependencies for a chart	<code>helm dependency build ./my-chart</code>
<code>helm dependency list</code>	Lists all dependencies for a chart	<code>helm dependency list ./my-chart</code>

5.4 Inspecting and testing

Command	Description	Example
<code>helm get all</code>	Gets all information for a release, values and templates included	<code>helm get all my-release</code>
<code>helm get values</code>	Displays the values used in a release	<code>helm get values my-release</code>
<code>helm test</code>	Runs the tests defined in a chart	<code>helm test my-release</code>
<code>helm template --debug</code>	Renders manifests and includes debug output	<code>helm template my-release ./my-chart --debug</code>
<code>helm install --dry-run</code>	Simulates an installation without performing it	<code>helm install my-release stable/nginx --dry-run</code>
<code>helm upgrade --dry-run</code>	Simulates an upgrade without applying it	<code>helm upgrade my-release stable/nginx --dry-run</code>

5.5 Values and customisation

Command	Description	Example
<code>helm install --values</code>	Installs a chart with custom values	<code>helm install my-release stable/nginx --values values.yaml</code>
<code>helm upgrade --values</code>	Upgrades a release with custom values	<code>helm upgrade my-release stable/nginx --values values.yaml</code>
<code>helm install --set</code>	Installs a chart with a value set on the command line	<code>helm install my-release stable/nginx --set replicaCount=3</code>
<code>helm upgrade --set</code>	Upgrades a release with a value set on the command line	<code>helm upgrade my-release stable/nginx --set replicaCount=5</code>
<code>helm uninstall --purge</code>	Removes a release and its resources, release history included	<code>helm uninstall my-release --purge</code>

5.6 Namespaces and contexts

Command	Description	Example
<code>helm list --namespace</code>	Lists releases in a specific namespace	<code>helm list --namespace kube-system</code>
<code>helm list -n</code>	Short form of the same thing	<code>helm list -n kube-system</code>
<code>helm install --namespace</code>	Installs a chart into a specific namespace	<code>helm install my-release stable/nginx --namespace mynamespace</code>

Command	Description	Example
<code>helm upgrade --namespace</code>	Upgrades a release in a specific namespace	<code>helm upgrade my-release stable/nginx --namespace mynamespace</code>
<code>helm uninstall --namespace</code>	Uninstalls a release from a specific namespace	<code>helm uninstall my-release --namespace kube-system</code>
<code>helm install --kube-context</code>	Installs a chart into the cluster named by a kubeconfig context	<code>helm install my-release stable/nginx --kube-context my-cluster</code>
<code>helm upgrade --kube-context</code>	Upgrades a release in a specific context	<code>helm upgrade my-release stable/nginx --kube-context my-cluster</code>

5.7 Repositories and publishing

Command	Description	Example
<code>helm repo remove</code>	Removes a chart repository	<code>helm repo remove stable</code>
<code>helm repo index</code>	Creates or updates the index file for a chart repository	<code>helm repo index ./charts</code>
<code>helm package --sign</code>	Packages a chart and signs it with a GPG key	<code>helm package ./my-chart --sign --key my-key-id</code>
<code>helm create --starter</code>	Creates a new chart from a starter template	<code>helm create --starter https://github.com/helm/charts.git</code>
<code>helm push</code>	Pushes a chart to a chart repository	<code>helm push ./my-chart my-repo</code>

5.8 History and rollbacks

Command	Description	Example
<code>helm rollback</code>	Rolls a release back to a previous version	<code>helm rollback my-release 1</code>
<code>helm history</code>	Displays the history of a release	<code>helm history my-release</code>
<code>helm history --max</code>	Limits the number of versions shown in the history	<code>helm history my-release --max 5</code>
<code>helm rollback --recreate-pods</code>	Rolls back to a previous version and recreates the pods	<code>helm rollback my-release 2 --recreate-pods</code>

6. Terraform

Terraform builds cloud infrastructure from code. Instead of clicking through an AWS, GCP or Azure console, servers and services are declared in configuration files and Terraform reconciles reality with them.

Command	Description
<code>terraform --help</code>	Display general help for the Terraform CLI
<code>terraform init</code>	Initialise the working directory and download the provider plugins the configuration needs
<code>terraform validate</code>	Check the configuration files for syntax errors and internal inconsistencies
<code>terraform plan</code>	Produce an execution plan showing what Terraform will do to reach the desired state
<code>terraform apply</code>	Apply the changes required to reach the desired state, prompting for approval first
<code>terraform show</code>	Display the state, or a saved plan, in human-readable form
<code>terraform output</code>	Display the output values defined in the configuration
<code>terraform destroy</code>	Destroy the infrastructure the configuration manages, prompting for confirmation
<code>terraform refresh</code>	Update the state file from the real infrastructure without applying any change
<code>terraform taint</code>	Mark a resource for recreation on the next apply, even if nothing about it changed
<code>terraform untaint</code>	Remove the tainted status from a resource
<code>terraform state</code>	Manage the state file: move resources between modules, remove them, inspect them
<code>terraform state list</code>	List every resource tracked in the state file
<code>terraform import</code>	Bring existing infrastructure under Terraform management
<code>terraform graph</code>	Generate a graph of the resources and their dependencies
<code>terraform providers</code>	List the providers the current configuration requires

On the backend: The backend is configured in a `backend` block inside `terraform { }`, not from the command line. It decides where the state file lives: locally by default, or remotely in S3, Azure Blob Storage, Terraform Cloud and others. `terraform init` is what reads it and migrates existing state.